
Camunda 7 Demo Layihəsi

Arxitektura, proses modeli və 8 mövzunun
ən xırda detallarına qədər texniki izahı

Message Correlation

DMN

Saga / Compensation

Async & Incidents

Migration

External Task

Kafka + Outbox

OAuth2 / OIDC

Java 21 · Spring Boot 4.1 · Camunda Platform 7.24
PostgreSQL · Apache Kafka · Keycloak · Thymeleaf
32 test · 13 test klasi · 0 failure

github.com/elkhanismayil/camunda-7-demo

Mündəricat

1. Layihə haqqında

Nə edir və niyə belə qurulub

Arxitektura

Repo strukturu

2. Camunda 7 əsasları — bu layihədə

Embedded engine vs standalone server

Engine-in verilənlər bazası cədvəlləri

Deployment necə baş verir

3. Proses modeli — element-element

4. `correlationId` və Message Correlation

5. Bənd 1 — Testing (`camunda-bpm-assert`)

6. Bənd 2 — DMN qərar cədvəli

7. Bənd 3 — Saga və Compensation

8. Bənd 4 — Engine daxili: async, retry, incident

9. Bənd 5 — Versiyalaşma və migration

10. Bənd 6 — External Task pattern

11. Bənd 7 — Kafka və Transactional Outbox

12. Bənd 8 — OAuth2 / OIDC

13. Yol boyu qarşılaşdığım real problemlər

14. Müsahibə üçün sual-cavab

15. İşə salma təlimatı

1. Layihə haqqında

1.1 Nə edir və niyə belə qurulub

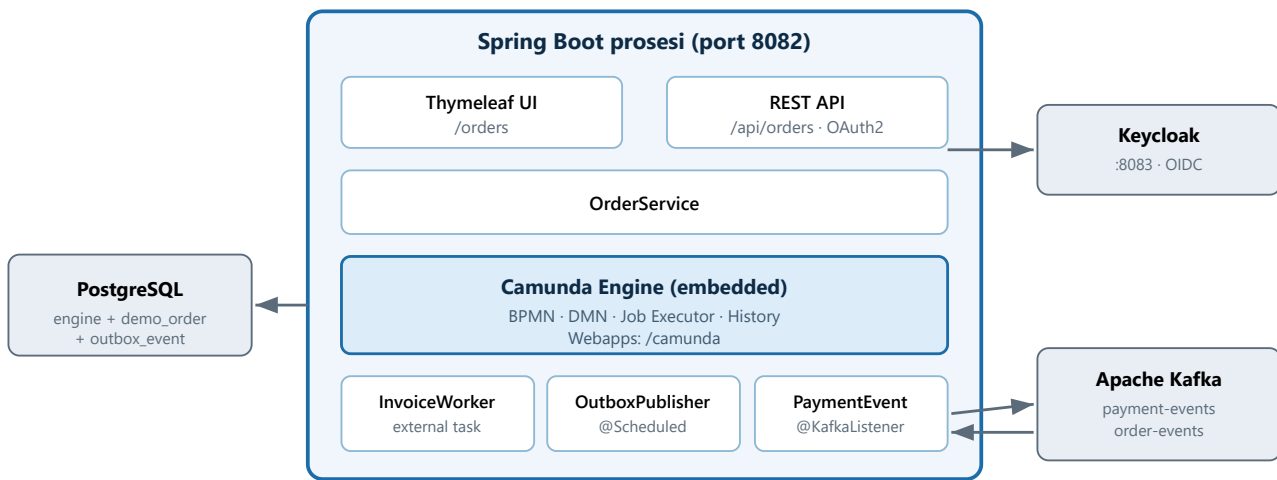
Layihə bir **sifariş-ödəniş prosesini** modelləşdirir. Sifariş yaradılır, riski qiymətləndirilir, ödəniş gözlənilir — və ya vaxt bitir. Ödəniş gələndə çatdırılma bildirişi göndərilir, invoice yaradılır, proses bitir. Çatdırılma uğursuz olarsa ödəniş geri qaytarılır.

Bu ssenari qəsdən *kiçik* seçilib: məqsəd çox funksiya deyil, bir dənə real axının üzərində Camunda-nın bütün ciddi mexanizmlərini göstərməkdir. Hər mexanizm süni bir "demo addımı" kimi deyil, həmin axının təbii hissəsi kimi oturur.

Əsas prinsip

UI (Thymeleaf) və REST API **eyni** `OrderService` -i çağırır. Yəni brauzerdə klikləmək paralel bir kod yolunu deyil, real engine-i işlədir. Demo ilə "əsl" kod arasında fərq yoxdur.

1.2 Arxitektura



Xarici asılılıq yalnız 3 konteynerdir. Engine **tətbiqin içindədir** — ayrıca Camunda serveri yoxdur.

Şəkil 1 — Ümumi arxitektura. Engine ayrıca server deyil, Spring Boot prosesinin içində kitabxana kimi işləyir.

1.3 Repo strukturu

TREE

```
src/main/java/az/company/camunda/
├─ Camunda7DemoApplication.java      // @EnableScheduling
├─ order/
│  ├─ Order.java   OrderStatus.java   OrderRepository.java
│  ├─ OrderService.java                // UI + REST üçün ortaqlar
│  ├─ OrderController.java              // /api/orders
│  ├─ OrderViewController.java          // /orders (Thymeleaf)
│  ├─ CreateOrderDelegate.java          MarkOrderPaidDelegate.java
│  ├─ CancelOrderDelegate.java          RevertPaymentDelegate.java
│  ├─ NotifyShippingDelegate.java       // BpmnError vs Exception
│  └─ InvoiceWorker.java                 // external task worker
├─ events/
│  ├─ OutboxEvent.java   OutboxEventRepository.java
│  ├─ OrderEventOutbox.java                // tranzaksiya daxilində yazır
│  ├─ OutboxPublisher.java                 // commit-dən sonra Kafka-ya
│  ├─ PaymentEventConsumer.java           // Kafka → correlation
│  └─ KafkaConfig.java                   // topic-lər, DLT, error handler
├─ security/
│  ├─ SecurityConfig.java                 // 3 filter chain
│  └─ KeycloakRealmRoleConverter.java
├─ src/main/resources/
│  ├─ order-payment-correlation.bpmn      // əsas proses
│  ├─ payment-risk-assessment.dmn        // qərar cədvəli
│  ├─ application.yaml
│  └─ templates/orders/index.html
├─ keycloak/realm-camunda-demo.json      // realm import
└─ docker-compose.yml                   // Kafka + Keycloak
```

2. Camunda 7 əsasları — bu layihədə

2.1 Embedded engine vs standalone server

Camunda 7-ni işə salmağın iki yolu var və bu, müsahibədə tez-tez qarışdırılır:

Yanaşma	Necə işləyir	Nə vaxt
Embedded <i>bu layihə</i>	Engine sizin tətbiqin JVM-ində kitabxana kimi qalxır. <code>RuntimeService</code> , <code>TaskService</code> adi Spring bean-ləridir. Delegate-lər sizin bean-lərinizdir. Engine sizin tranzaksiyanızı paylaşır.	Bir tətbiq bir prosesə sahibdir. Mikroservis arxitekturası.
Standalone	Ayrıca server (Tomcat/WildFly). Tətbiqlər ona REST ilə müraciət edir. Delegate-lər serverin classpath-ində olmalıdır — ya da external task istifadə edilir.	Bir çox komanda bir engine-i paylaşır. Mərkəzi BPM platforması.

BUILD.GRADLE

```
// Bu bir sətir engine-i tətbiqin içinə gətirir.  
implementation "org.camunda.bpm.springboot:camunda-bpm-spring-boot-starter-webapp:7.24.0"
```

Diqqət: paylaşılan tranzaksiya

Embedded rejimdə `orderRepository.save(...)` ilə engine-in öz yazıları **eyni tranzaksiyadadır**. Bu, Bənd 7-dəki outbox pattern-inin işləməsinin səbəbidir — və eyni zamanda delegate-də atılan istisnanın proses addımını da geri qaytarmasının səbəbi.

2.2 Engine-in verilənlər bazası cədvəlləri

Camunda öz sxemini avtomatik yaradır (`camunda.bpm.database.schema-update: true`). Debug edərkən bilməli olduğunuz cədvəllər:

Cədvəl	Prefiks	Nə saxlayır
<code>act_re_procdef</code>	RE = repository	Deploy olunmuş proses tərifləri. <code>key_</code> , <code>version_</code> , <code>deployment_id_</code> . <i>Bənd 5 burada oxunur.</i>
<code>act_re_deployment</code>	RE	Hər deployment bir sətir.
<code>act_ru_execution</code>	RU = runtime	İşləyən instansiyalar və onların execution ağacı. <code>proc_def_id_</code> , <code>business_key_</code> .
<code>act_ru_variable</code>	RU	Proses dəyişənləri — <code>correlationId</code> , <code>amount</code> , <code>riskLevel</code> burada.
<code>act_ru_task</code>	RU	Aktiv user task-lər.
<code>act_ru_job</code>	RU	Timer və async job-lər. <code>retries_</code> sütunu <i>Bənd 4</i> -ün mövzudur.
<code>act_ru_incident</code>	RU	Retry-ları bitmiş job-lər. <code>incident_type_ = failedJob</code> .
<code>act_ru_ext_task</code>	RU	External task-lər. <code>topic_name_</code> , <code>worker_id_</code> , <code>lock_exp_time_</code> . <i>Bənd 6.</i>
<code>act_ru_event_subscr</code>	RU	Mesaj/signal abunəlikləri. <code>PaymentReceived</code> gözləyən instansiyalar burada.
<code>act_hi_*</code>	HI = history	Bitmiş hər şeyin arxivi. <code>historyTimeToLive</code> nə qədər saxlanacağını deyir.

historyTimeToLive məcburidir

Camunda 7.20-dən sonra `historyTimeToLive` təyin edilməyən proses deploy olunmur. Bu layihədə hər BPMN və DMN-də `camunda:historyTimeToLive="180"` var. Səbəb: TTL-siz tarixçə cədvəlləri sonsuz böyüyür və bir gün bazanı doldurur.

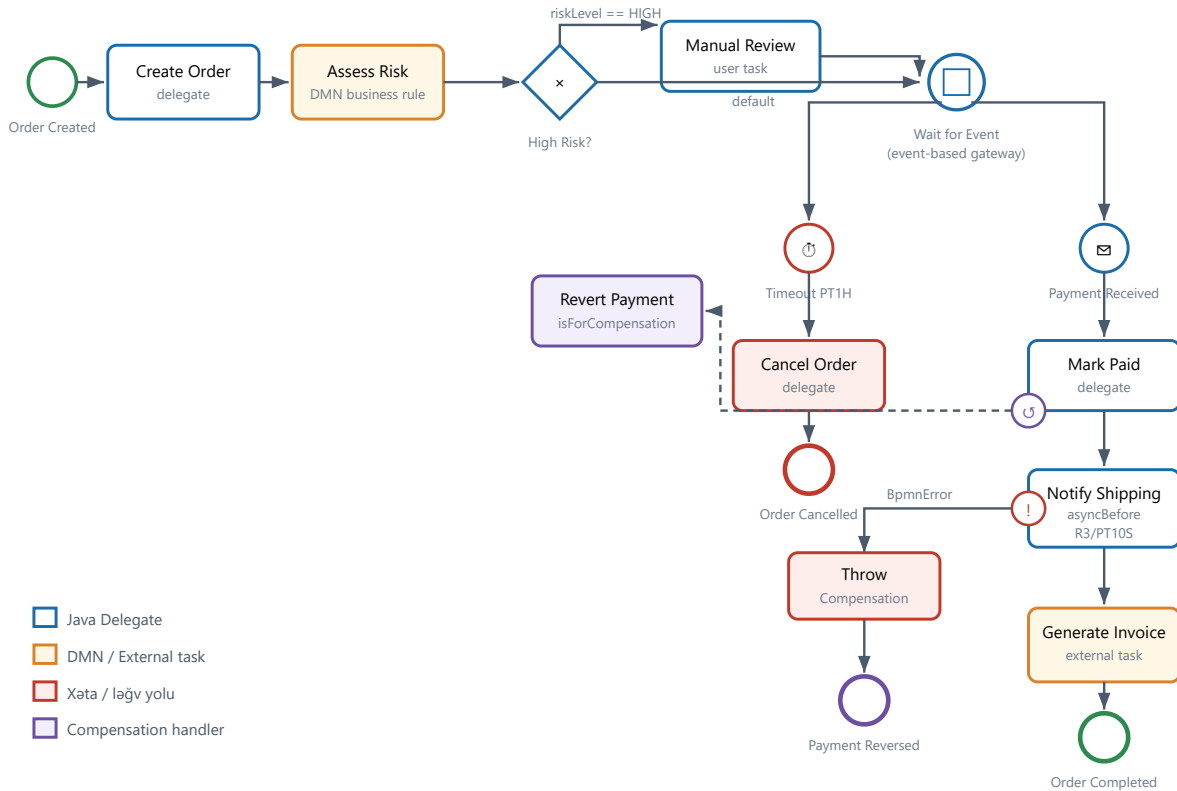
2.3 Deployment necə baş verir

Spring Boot starter startup-da `classpath*:**/*.bpmn`, `**/*.dmn` pattern-i ilə skan edir və tapdığı hər şeyi deploy edir. Fayl dəyişməyibsə yeni versiya yaratmır (checksum müqayisəsi).

Bu avtomatik skan real problem yaraddı

Bənd 5-də (migration) v1/v2/v3 modellərini test resource-larına fayl kimi qoysaydım, **üçü də context qalxanda deploy olundu** və versiyalaşmanı müşahidə etmək mümkün olmazdı. Ona görə o testdə modellər BPMN Model API ilə kodda qurulur.

3. Proses modeli — element-element



Şəkil 2 — `order-payment-correlation.bpmn` tam axını. Kəşik xətt kompensasiya bağlantısıdır (sequence flow deyil).

Element-element

Element	Tip	Rolu
StartEvent_OrderCreated	Start event	Proses <code>startProcessInstanceByKey</code> ilə başlayır, business key = <code>correlationId</code> .
Task_CreateOrder	Service (delegate)	<code>demo_order</code> cədvəlinə satış yazır.
Task_AssessRisk	Business rule	DMN cədvəlini işlədir, nəticəni <code>riskLevel</code> dəyişəninə yazır.
Gateway_RiskCheck	Exclusive gateway	<code>riskLevel == "HIGH"</code> → review; əks halda default axın.
Task_ManualReview	User task	<code>candidateGroups="risk-review"</code> . Tasklist-də görünür.
Gateway_WaitForEvent	Event-based gateway	İki hadisə arasında yarış: mesaj, yoxsa timer.
Event_PaymentReceived	Message catch	<code>PaymentReceived</code> mesajını gözləyir.
Event_PaymentTimeout	Timer catch	<code>PT1H</code> — bir saat.
Task_MarkPaid	Service (delegate)	Status → PAID, outbox-a <code>ORDER_PAID</code> yazır. Kompensasiya olunandır.
Task_NotifyShipping	Service (delegate)	<code>asyncBefore</code> + <code>R3/PT10S</code> . Biznes xətası → kompensasiya; texniki xəta → retry.
Task_GenerateInvoice	Service (external)	Topic <code>invoice-generation</code> . Wait state.
Task_RevertPayment	Compensation handler	<code>isForCompensation="true"</code> — normal axında <i>heç vaxt</i> işləmir.

4. `correlationId` və Message Correlation

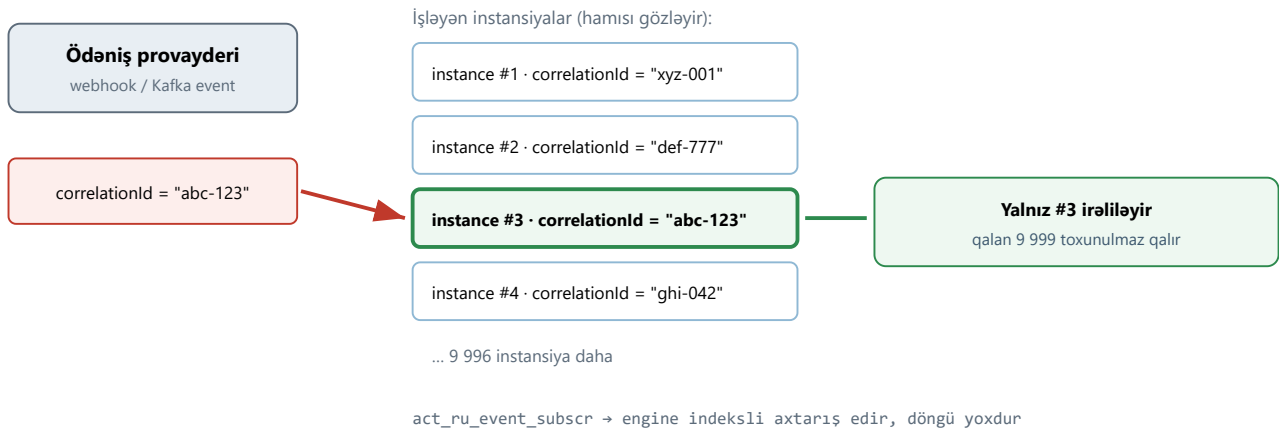
Problem

Fərz edin ki, eyni anda 10 000 sifariş prosesi işləyir və hər biri öz ödənişini gözləyir. Ödəniş provayderindən webhook gəlir: "filan sifariş ödənildi". Sual: **bu bildiriş 10 000 instansiyadan hansına aiddir?**

Naiv cavab — process instance id saxlamaq. Amma xarici sistem sizin engine-inizin daxili id-lərini bilmir və bilməməlidir. Bizim tərəfdə isə həmin id yalnız proses başladıqdan sonra yaranır.

Həll: biznes açarı ilə korrelyasiya

`correlationId` — **bizim** yaratdığımız biznes identifikatorudur. Həm `demo_order` cədvəlində sütun, həm proses dəyişəni, həm də BPMN business key kimi saxlanılır. Xarici sistem yalnız bunu bilir.



Şəkil 3 — Message correlation: biznes açarı minlərlə gözləyən instansiyadan tam birini tapır.

Kod

ORDERSERVICE.JAVA

```
public ProcessInstance createOrder(String customerName, BigDecimal amount) {
    String correlationId = UUID.randomUUID().toString(); // biznes açarı BİZ yaradıırıq

    Map<String, Object> variables = new HashMap<>();
    variables.put("correlationId", correlationId);
    variables.put("customerName", customerName);
    variables.put("amount", amount);

    // 2-ci argument = business key. Cockpit-də axtarış üçün, həm də
    // processInstanceBusinessKey() sorğuları üçün.
    return runtimeService.startProcessInstanceByKey(PROCESS_KEY, correlationId, variables);
}

public boolean notifyPaymentReceived(String correlationId) {
    try {
        runtimeService.createMessageCorrelation("PaymentReceived")
            .processInstanceVariableEquals("correlationId", correlationId)
            .correlateWithResult();
        return true;
    } catch (MismatchingMessageCorrelationException e) {
        // Ya belə instansiya yoxdur, ya da o, hazırda mesaj gözləmir.
        // Bu fərq Bənd 7-də həlledici olur.
        return false;
    }
}
```

Korrelyasiya metodlarının fərqi

Metod	Davranış
<code>correlate()</code>	Tam bir instansiya tapmalıdır. Tapmazsa və ya birdən çox taparsa istisna atır.
<code>correlateWithResult()</code>	Eyni, amma nəticə obyektini qaytarır (hansı instansiya, hansı tip).
<code>correlateAll()</code>	Uyğun gələn hamısını irəlilədir. Broadcast üçün.
<code>correlateExclusively()</code>	Pessimistik kilid alır. Eyni instansiya paralel mesajlar gələndə optimistic locking istisnalarının qarşısını alır.

Testdə tapdığım incəlik

Event-based gateway arxasında park olunmuş execution-lar ActivityInstance ağacında **catch event-in deyil, gateway-in öz activity id-si** ilə görünür. Yəni `isWaitingAt("Event_PaymentReceived")` yox, `isWaitingAt("Gateway_WaitForEvent")` yazmaq lazımdır. Bunu ağacı çap edərək tapdım — sənəddə açıq yazılmır.

ORDERPAYMENTCORRELATIONTEST.JAVA – MAHIYYƏT

```
ProcessInstance instanceA = start(correlationIdA); // Alice
ProcessInstance instanceB = start(correlationIdB); // Bob

assertThat(instanceA).isWaitingAt("Gateway_WaitForEvent");
assertThat(instanceB).isWaitingAt("Gateway_WaitForEvent");

runtimeService.createMessageCorrelation("PaymentReceived")
    .processInstanceVariableEquals("correlationId", correlationIdA)
    .correlateWithResult();

// A irəlilədi, B toxunulmadı – korrelyasiyanın bütün mahiyyəti budur.
assertThat(instanceB).isNotEnded().isWaitingAt("Gateway_WaitForEvent");
assertThat(orderRepository.findByCorrelationId(correlationIdB).orElseThrow().getStatus())
    .isEqualTo(OrderStatus.AWAITING_PAYMENT);
```

5. Bənd 1 — Testing (camunda-bpm-assert)

Prosesi test etmək adı unit testdən fərqlidir: sınağınız şey bir metodun qaytardığı dəyər deyil, **token-in modeldə hansı yolu getməsidir**. `camunda-bpm-assert` məhz bunun üçün fluent API verir.

BUILD.GRADLE

```
testImplementation "org.camunda.bpm:camunda-bpm-assert:7.24.0"
```

Əsas assertion-lar

Assertion	Nə yoxlayır
<code>assertThat(pi).isEnded()</code>	İnstansiya bitib.
<code>assertThat(pi).isNotEnded()</code>	Hələ işləyir.
<code>assertThat(pi).isWaitingAt("Id")</code>	Göstərilən activity-də dayanıb.
<code>assertThat(pi).hasPassed("A", "B")</code>	Bu activity-lərdən <i>keçib</i> (tarixçəyə baxır).
<code>assertThat(pi).hasVariables("x")</code>	Dəyişən mövcuddur.
<code>assertThat(task).hasCandidateGroup("g")</code>	User task-ın qrupu.

Determinizm — ən vacib qərar

Test profilində job executor söndürülüb

APPLICATION-TEST.YAML

```
camunda.bpm:
  job-execution:
    enabled: false # testlər job-ları özləri işə salır
```

Səbəb: job executor arxa fon thread-idir. Açıq qalsa test ilə onun arasında yarış olur — bəzən keçən, bəzən keçməyən test alırsınız. Söndürüb job-ları əl ilə icra etmək testi **tam determinik** edir:

JAVA

```
Job job = managementService.createJobQuery().processInstanceId(pi.getId()).singleResult();
managementService.executeJob(job.getId()); // indi, bu thread-də, sinxron
```

Timer-i "gözləmədən" sınamaq

Timeout `PT1H`-dir. Testdə bir saat gözləmək absurd olardı. `ClockUtil` engine-in saat mənbəyini əvəz edir.

ORDERPAYMENTTIMEOUTTEST.JAVA

```
// Engine-in "indi"sini bir saat bir dəqiqə irəli çəkirik.
ClockUtil.setCurrentTime(Date.from(Instant.now().plus(Duration.ofHours(1).plusMinutes(1))));

Job timeoutJob = managementService.createJobQuery()
    .processInstanceId(instance.getId()).singleResult();
managementService.executeJob(timeoutJob.getId());

assertThat(instance).isEnded().hasPassed("Task_CancelOrder", "Event_OrderCancelled");

// @AfterEach-də mütləq: ClockUtil.reset() - yoxsa növbəti testlər zəhərlənir.
```

Paylaşılan Spring context-ə diqqət

Bütün testlər eyni H2 bazasını paylaşır. Bənd 5-in testi deployment yaradır, ona görə `@AfterEach`-də `repositoryService.deleteDeployment(id, true)` (cascade) çağırılır. Əks halda versiya nömrələri testdən testə sürüşər və testlər sıraya həssas olardı.

6. Bənd 2 — DMN qərar cədvəli

Niyə DMN, if-else deyil?

Risk hədləri biznes qaydasıdır. Onlar dəyişəndə (və dəyişəcəklər) kodun yenidən kompilyasiya olunması, review, deploy tələb olunmamalıdır. DMN cədvəli **biznes analitikin** oxuya və redaktə edə biləcəyi artefaktdır — və Camunda onu ayrıca versiyalaşdırır.

Cədvəl

Amount (input)	riskLevel (output)	Prosesə təsiri
< 100	LOW	Birbaşa ödəniş gözləməyə
[100..1000)	MEDIUM	Birbaşa ödəniş gözləməyə
>= 1000	HIGH	Manual review user task

Bu layihədə real dərs

İlk versiyada DMN `riskLevel` hesablayırdı — və **heç nə etmirdi**. Dəyər hesablanıb dəyişəndə qalırdı, proses ona görə dəyişmirdi. Bu, "DMN əlavə etdim" deyib əslində heç nə göstərməyən demo idi. Exclusive gateway və manual review task əlavə etdikdən sonra DMN-in nəticəsi *həqiqətən* axını idarə etməyə başladı. Müsahibədə DMN-dən danışsınızsa, mütləq göstərin ki, çıxış *nəyisə* dəyişir.

DMN faylı

PAYMENT-RISK-ASSESSMENT.DMN

```
<decision id="Decision_PaymentRisk" name="Payment Risk Assessment"
  camunda:historyTimeToLive="180">
  <decisionTable id="DecisionTable_PaymentRisk" hitPolicy="UNIQUE">

    <input id="Input_Amount" label="Order Amount">
      <inputExpression typeRef="double"><text>amount</text></inputExpression>
    </input>

    <output id="Output_RiskLevel" name="riskLevel" typeRef="string"/>

    <rule id="Rule_High">
      <inputEntry><text>>= 1000</text></inputEntry>
      <outputEntry><text>"HIGH"</text></outputEntry>
    </rule>
  </decisionTable>
</decision>
```

Hit policy — müsahibə sualı

Policy	Mənası
UNIQUE	Yalnız bir qayda uyğun gəlməlidir. İki qayda uyğun gələrsə istisna . Bizim seçim — hədlər üst-üstə düşərsə bunu dərhal bilmək istəyirik.
FIRST	İlk uyğun qayda. Sıra vacib olur, gizli səhvlər asanlaşır.
COLLECT	Bütün uyğun nəticələr siyahı kimi.
PRIORITY	Output dəyərlərinin təyin olunmuş prioritetinə görə.
ANY	Çoxu uyğun gələ bilər, amma hamısı eyni nəticəni verməlidir.

BPMN-ə bağlanması

ORDER-PAYMENT-CORRELATION.BPMN

```
<bpmn:businessRuleTask id="Task_AssessRisk" name="Assess Payment Risk"
  camunda:decisionRef="Decision_PaymentRisk"
  camunda:resultVariable="riskLevel"
  camunda:mapDecisionResult="singleEntry">
```

mapDecisionResult	Nəticə tipi
singleEntry	Tək dəyər (bizdə: "HIGH" stringi). Bir sətir, bir sütun.
singleResult	Map — bir sətir, çox sütun.
collectEntries	List — çox sətir, bir sütun.
resultList	List<Map> — çox sətir, çox sütun.

Gateway — DMN nəticəsinin işə düşdüyü yer

BPMN

```
<bpmn:exclusiveGateway id="Gateway_RiskCheck" name="High Risk?"
    default="Flow_SkipReview"/> <!-- default MÜTLƏQ olmalıdır -->

<bpmn:sequenceFlow id="Flow_ToManualReview" ...>
  <bpmn:conditionExpression>${riskLevel == "HIGH"}</bpmn:conditionExpression>
</bpmn:sequenceFlow>
```

default axın olmasa

Heç bir şərt ödənmirsə engine `ProcessEngineException: No outgoing sequence flow ... could be selected` atır və instansiya incident-ə düşür. Exclusive gateway-də default axın demək olar ki, həmişə lazımdır.

DMN-i BPMN-siz test etmək

PAYMENTRISKASSESSMENTDMNTEST.JAVA

```
DmnDecisionTableResult result = decisionService.evaluateDecisionTableByKey(
    "Decision_PaymentRisk", Variables.putValue("amount", 50000));

assertThat(result.getSingleEntry()).isEqualTo("HIGH");

// Sərhəd dəyərləri ayrıca yoxlanılır: 99.99 / 100 / 999.99 / 1000
// Bunlar məhz cədvəldə səhv yazıla bilən yerlərdir.
```

Doğru import

`org.camunda.bpm.dmn.engine.DmnDecisionTableResult` — `org.camunda.bpm.engine.dmn.*` deyil. Bu, məni bir dəfə yandırdı.

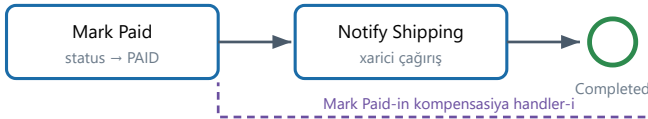
7. Bənd 3 — Saga və Compensation

Problem

Ödəniş qeydə alınıb (PAID), sonra çatdırılma provayderi sifariş rədd edir. İndi sistemdə **uyğunsuz vəziyyət** var: müştəri ödəyib, amma heç nə göndərilməyəcək.

Bu, klassik distributed transaction problemidir. İki fərqli sistem (ödəniş + çatdırılma) arasında ACID tranzaksiya yoxdur. **Saga** pattern-i cavabdır: hər addımın *kompensasiya edən* əks addımı olur, və uğursuzluqda artıq icra olunmuş addımlar tərs sıra ilə geri alınır.

Normal axın



Biznes xətası baş verir



Nəticə

Sifariş PAID-də ilişib qalmır.
REFUNDED olur — yəni sistem
özünü uyğun vəziyyətə gətirir.
*Bu, rollback DEYİL —
biznes səviyyəsində əks əməliyyat.*

Şəkil 4 — Saga: uğursuzluqda artıq commit olunmuş addım əks əməliyyatla geri alınır.

Kompensasiya ≠ rollback

Database rollback commit olunmamış dəyişikliyi silir və heç bir iz qalmır. Kompensasiya isə **artıq commit olunmuş və dünyaya görünmüş** addımı əks əməliyyatla balanslaşdırır. Ona görə status `AWAITING_PAYMENT` -ə qayıtmır — `REFUNDED` olur. Fərq auditdə görünür və görünməlidir.

BPMN quruluşu — 4 element

ORDER-PAYMENT-CORRELATION.BPMN

```
<!-- 1. Kompensasiya olunan task-a bağlı boundary event -->
<bpmn:boundaryEvent id="Boundary_MarkPaidCompensation" attachedToRef="Task_MarkPaid">
  <bpmn:compensateEventDefinition/>
</bpmn:boundaryEvent>

<!-- 2. Handler. isForCompensation -> normal axında HEÇ VAXT işləmir -->
<bpmn:serviceTask id="Task_RevertPayment" name="Revert Payment"
  isForCompensation="true"
  camunda:delegateExpression="{revertPaymentDelegate}"/>

<!-- 3. Tetikləyici: boundary error event -> throw compensation -->
<bpmn:intermediateThrowEvent id="Event_ThrowCompensation">
  <bpmn:compensateEventDefinition/>
</bpmn:intermediateThrowEvent>

<!-- 4. Bağlantı. DİQQƏT: bu, <bpmn:process>-in SON uşağı olmalıdır -->
<bpmn:association sourceRef="Boundary_MarkPaidCompensation"
  targetRef="Task_RevertPayment"/>
```

XSD tələsi — bir saatımı yedi

BPMN sxemində `tProcess` tipi **bütün flowElement-ləri artifact-lardan əvvəl** tələb edir. `<bpmn:association>` artifact-dır. Onu sequence flow-ların arasına qoysanız deployment belə bir xəta ilə çökür:

STDERR

```
cvc-complex-type.2.4.a: Invalid content was found starting with
element 'bpmn:serviceTask'. One of '{artifact}' is expected.
```

Həll: association-ı `</bpmn:process>` -dən dərhal əvvəllə köçürmək.

Biznes xətası vs texniki xəta — bənd 3-ün əsl dərsi

NOTIFYSHIPPINGDELEGATE.JAVA

```
@Override
public void execute(DelegateExecution execution) {
    String customerName = (String) execution.getVariable("customerName");
    if (customerName == null) return;

    if (customerName.contains("ShipFail")) {
        // BİZNES nəticəsi: provayder sifarişini rədd etdi. Bu, modelləşdirilmiş
        // haldır → boundary error event tutur → kompensasiya.
        throw new BpmnError(SHIPPING_FAILED_ERROR_CODE,
            "Shipping provider rejected the order for " + customerName);
    }

    if (customerName.contains("TechFail")) {
        // TEXNİKİ nasazlıq: şəbəkə qırılıb. Bu, modelləşdirilmiş hal DEYİL →
        // engine retry edir → retry-lar bitəndə incident.
        throw new IllegalStateException("Shipping provider unreachable for " + customerName);
    }
}
```

Atılan	Engine nə edir	Sifarişin sonu
BpmnError	Boundary error event tutur → kompensasiya işə düşür	REFUNDED
RuntimeException	R3/PT10S üzrə retry, sonra incident	PAID qalır, proses park olunur

Niyə bu fərq kritikdir

Rədd edilmiş çatdırılma modelləşdirilmiş biznes nəticəsidir — kompensasiya doğrudur. Qırılmış bağlantı isə deyil. Onu kompensasiya etmək o deməkdir ki, şəbəkə problemi ucbatından **ödəmiş müştərinin pulunu səssizcə geri qaytarırsınız**. Doğru davranış retry, sonra isə operatorun Cockpit-də müdaxiləsidir.

8. Bənd 4 — Engine daxili: async, retry, incident

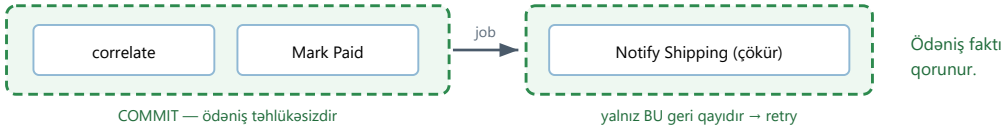
asyncBefore nədir?

Camunda default olaraq bir proses instansiyasını **bir tranzaksiyada** mümkün qədər irəli aparır — növbəti wait state-ə qədər. Yəni ödəniş korrelyasiyası ilə çatdırılma bildirişi *eyni* tranzaksiyada olurdu. Çatdırılma çağırışı çökməyə, tranzaksiya geri qaydır və **ödənişin qeydə alınması da itir**.

asyncBefore OLMADAN — bir tranzaksiya



asyncBefore İLƏ — iki tranzaksiya



Şəkil 5 — `asyncBefore` tranzaksiya sərhədidir: engine burada commit edir və qalanını job-a köçürür.

BPMN

```
<bpmn:serviceTask id="Task_NotifyShipping"
  camunda:delegateExpression="${notifyShippingDelegate}"
  camunda:asyncBefore="true" <!-- tranzaksiya sərhədi -->
  camunda:failedJobRetryTimeCycle="R3/PT10S"> <!-- 3 dəfə, 10 san ara -->
```

Retry → incident yolu

Cəhd	<code>act_ru_job.retries_</code>	Nə baş verir
başlanğıc	3	Job yaradılır
1	3 → 2	Xəta, 10 saniyə sonra yenidən
2	2 → 1	Xəta, 10 saniyə sonra yenidən
3	1 → 0	Xəta → <code>act_ru_incident</code> -ə sətir , tip <code>failedJob</code>

Incident yaranandan sonra job daha icra olunmur. Proses **ölmür** — park olunur və operator Cockpit-də səbəbi düzəldib retry-ları sıfırlayana qədər gözləyir. Bu, "səssiz uğursuzluq" ilə "gözlə görünən uğursuzluq" arasındakı fərqdır.

SHIPPINGRETRYANDINCIDENTTEST.JAVA – MAHİYYƏT

```
while (attempts < MAX_ATTEMPTS && retriesLeft(instance) > 0) {
  attempts++;
  try { managementService.executeJob(currentJob(instance).getId()); }
  catch (RuntimeException expected) { /* hər cəhd uğursuzdur */ }
}

Incident incident = runtimeService.createIncidentQuery()
  .processInstanceId(instance.getId()).singleResult();

assertThat(incident.getIncidentType()).isEqualTo("failedJob");
assertThat(incident.getActivityId()).isEqualTo("Task_NotifyShipping");

// Ən vacib assertion: texniki xəta prosesi öldürməməli və ödənişi
// səssizcə geri qaytarmamalıdır.
assertThat(statusOf(correlationId)).isEqualTo(OrderStatus.PAID);
```

Job executor tənzimləməsi — real trade-off-lar

APPLICATION.YAML

```
camunda.bpm:
  job-execution:
    core-pool-size: 3
    max-pool-size: 10
    queue-capacity: 20
    max-jobs-per-acquisition: 3
    lock-time-in-millis: 300000
    wait-time-in-millis: 5000
```

Parametr	Trade-off
<code>max-pool-size</code>	Job icra edən thread-lər. Az olsa throughput boğulur; çox olsa darboğaz DB connection pool-a keçir. Hikari pool ölçüsündən böyük olmamalıdır — əks halda thread-lər connection gözləyir.
<code>queue-capacity</code>	Alınmış, amma hələ icra olunmamış job-lar. Böyük növbə backpressure-i gizlədir və çökmə zamanı itən işin pəncərəsini uzadır.
<code>max-jobs-per-acquisition</code>	Bir dövrədə neçə job kilidlənir. Artırmaq DB gediş-gəlişini azaldır, amma klasterdə node-lar arasında optimistic locking rəqəbətini artırır.
<code>lock-time-in-millis</code>	Ən təhlükəli parametr. Ən yavaş real job-dan uzun olmalıdır . Qısa olsa, job hələ işləyərkən başqa node onu götürür və <i>iki dəfə icra olunur</i> .

Klaster davranışı — tez-tez soruşulur

Camunda 7-də job executor-lar mərkəzi koordinator olmadan işləyir: hər node `SELECT ... FOR UPDATE` bənzəri sorğu ilə job-ları kilidləyir. Konfliktlər `OptimisticLockingException` kimi görünür və **normaldır** — engine həmin dövrəni sadəcə buraxır. Log-da bunları görmək nasazlıq əlaməti deyil; çox görmək isə `max-jobs-per-acquisition`-in böyük olduğunu göstərir.

9. Bənd 5 — Versiyalaşma və migration

Əsas zamanət

Camunda-da `deploy` **heç vaxt köhnəni üzərinə yazmır**. Eyni process definition key ilə dəyişdirilmiş model deploy edəndə engine onu *yeni versiya* kimi saxlayır (`act_re_procdef.version_` artır), və artıq işləyən instansiyalar başladıkları definition-a bağlı qalır (`act_ru_execution.proc_def_id_` dəyişmir).

v1 deploy olunur, instansiya başlayır

Reserve Stock → Ship Order
version_ = 1 • token burada

v2 deploy olunur (araya Fraud Check əlavə olunub)

Reserve Stock → Fraud Check → Ship Order
version_ = 2

Nəticə: iki fərqli davranış

Köhnə instansiya v1-də qalır → Reserve Stock bitəndə Ship Order

Yeni instansiya v2-də başlayır → Reserve Stock bitəndə Fraud Check

Eyni klik, fərqli nəticə.

Migration: köhnə instansiyanı v2-yə köçürmək

```
createMigrationPlan(v1, v2)
.mapEqualActivities().build()
newMigration(plan).processInstanceIds(...).execute()
```

Token yenə **Reserve Stock**-da — irəliləmir, restart olmur.
Amma artıq v2 altında: növbəti addım Fraud Check.

Şəkil 6 — Deployment additivdir; instansiyaları yeni modelə keçirmək ayrıca, açıq əməliyyatdır.

Niyə bu belədir?

Bu, deployment-i **təhlükəsiz** edir — yeni model heç vaxt uçuşdakı işi korlaya bilməz. Qiyməti isə odur ki, köhnə instansiyalar köhnə modeli icra etməyə davam edir. 500 sifariş ödəniş gözləyirsə, onlar siz açıq şəkildə köçürənə qədər köhnə axınla gedəcək.

Versiya seçimi

Metod	Hansı versiya
<code>startProcessInstanceByKey(key)</code>	Həmişə ən sonuncu
<code>startProcessInstanceById(defId)</code>	Konkret versiya — id versiyaya bağlıdır

Migration kodu

JAVA

```
MigrationPlan plan = runtimeService.createMigrationPlan(v1.getId(), v2.getId())
    .mapEqualActivities() // id-ləri hər iki modeldə eyni olanları map edir
    .build();

runtimeService.newMigration(plan)
    .processInstanceIds(instance.getId())
    .execute();
```

Faydalı əlavələr:

Metod	Nə edir
<code>.mapActivities("Old", "New")</code>	Açıq map — id dəyişibsə məcburidir.
<code>.updateEventTrigger()</code>	Timer/mesaj abunəliyini yeni tərifi uyğun yenidən qurur. Timer müddəti dəyişibsə lazımdır.
<code>.skipCustomListeners()</code>	Migration zamanı execution listener-ləri işə salmır.
<code>.processInstanceQuery(query)</code>	Id siyahısı əvəzinə sorğu ilə toplu migration.
<code>.executeAsync()</code>	Batch kimi — minlərlə instansiya üçün.

Ən maraqlı hal: activity adı dəyişibsa

Plan qurulur, icra rədd olunur

`mapEqualActivities()` yalnız id-ləri *eyni* olanları map edir. Yeni versiyada instansiyanın dayandığı activity adlandırılıbsa:

- `build()` **problemsiz keçir** — çünki build yalnız *planı* validate edir, instansiyaları yox
- `execute()` `MigratingProcessInstanceValidationException` atır
- instansiya **toxunulmamış** qalır — engine token-in hara aid olduğunu təxmin etmir

PROCESSVERSIONİNGANDMIGRATIONTEST.JAVA

```
MigrationPlan naivePlan = runtimeService.createMigrationPlan(v1.getId(), v2.getId())
    .mapEqualActivities()
    .build(); // ← burada xəta VOXDUR

assertThatThrownBy(() -> runtimeService.newMigration(naivePlan)
    .processInstanceIds(instance.getId())
    .execute()) // ← xəta BURADA
    .isInstanceOf(MigratingProcessInstanceValidationException.class);

assertThat(definitionIdOf(instance))
    .as("rədd edilmiş migration instansiyanı olduğu kimi qoymalıdır")
    .isEqualTo(v1.getId());

// Həll retry deyil - açıq göstəriş:
MigrationPlan explicitPlan = runtimeService.createMigrationPlan(v1.getId(), v2.getId())
    .mapEqualActivities()
    .mapActivities("Task_ShipOrder", "Task_DispatchOrder")
    .build();
```

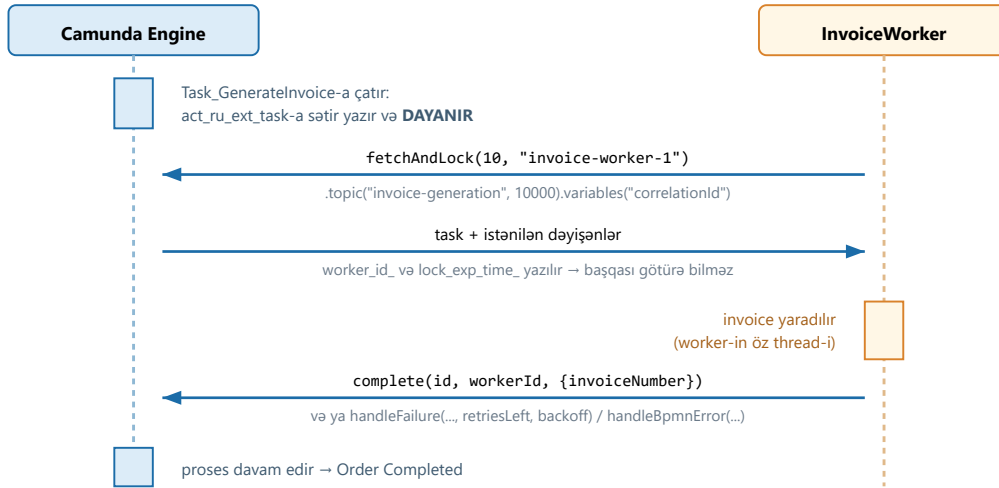
10. Bənd 6 — External Task pattern

İstiqamətin tərsinə çevrilməsi

Java delegate-i engine *çağırır*: engine-in öz tranzaksiyasında, job executor thread-ində, engine-in yükləyə bildiyi klasdan. External task əksinədir — BPMN **topic** adlandırır, bean yox. Engine bizim klasımızdan xəbərsizdir.

BPMN — BİZİM HEÇ BİR KLASIMIZIN ADI YOXDUR

```
<bpmn:serviceTask id="Task_GenerateInvoice" name="Generate Invoice"
camunda:type="external"
camunda:topic="invoice-generation">
```



Şəkil 7 — External task ardıcılığı. Bütün təşəbbüs worker-dədir; engine yalnız cavab verir.

Delegate vs External task

	Delegate (Task_NotifyShipping)	External (Task_GenerateInvoice)
Kim kimi çağırır	Engine bizim bean-i çağırır	Worker engine-dən iş istəyir
BPMN istinadı	<code>delegateExpression</code> → bean	<code>camunda:topic</code> → bizdən heç nə
Thread / tranzaksiya	Job executor thread-i, engine tranzaksiyası	Worker-in öz thread-i, öz tranzaksiyası
Xəta bildirişi	<code>BpmnError</code> / exception atmaqla	<code>handleBpmnError</code> / <code>handleFailure</code> çağırmaqla
Retry-a kim qərar verir	Model (<code>failedJobRetryTimeCycle</code>)	Worker , hər xəta üçün ayrıca
Worker sönülürsə	Job xəta verir, retry olunur	Heç nə olmur — instansiya gözləyir
Dil / deployment	JVM, engine ilə eyni classpath	İstənilən dil, ayrıca deployment

Worker kodu

INVOICEWORKER.JAVA

```

@Scheduled(fixedDelayString = "${demo.invoice-worker.poll-interval-ms:2000}")
public void pollForWork() {
    List<LockedExternalTask> tasks = externalTaskService
        .fetchAndLock(MAX_TASKS_PER_FETCH, WORKER_ID)
        .topic(TOPIC, LOCK_DURATION_MS)
        // Worker lazım olan dəyişənləri ADI ilə istəyir. Ona heç vaxt
        // baza bağlantısı verilmir – prosesə yeganə bağ bu payload-dır.
        .variables("correlationId")
        .execute();

    tasks.forEach(this::handle);
}

private void handle(LockedExternalTask task) {
    String correlationId = (String) task.getVariables().get("correlationId");
    try {
        String invoiceNumber = generateInvoice(correlationId);
        externalTaskService.complete(task.getId(), WORKER_ID,
            Map.of("invoiceNumber", invoiceNumber));
    } catch (RuntimeException e) {
        // Retry siyasəti BURADA yaşayır, modeldə deyil.
        Integer remaining = task.getRetries();
        int retriesLeft = remaining == null ? INITIAL_RETRIES : remaining - 1;

        externalTaskService.handleFailure(task.getId(), WORKER_ID,
            "Invoice generation failed", e.toString(), retriesLeft, RETRY_BACKOFF_MS);
    }
}
}

```

fetchAndLock — konkurentliyin bütün hekayəsi

On eyni worker eyni topic-i poll edə bilər. Lock onların eyni işi iki dəfə görməsinin qarşısını alır. Test bunu birbaşa göstərir: worker-1 götürəndən sonra worker-2 **boş siyahı** alır; `unlock()` -dən sonra işə götürə bilər.

Nəqliyyat qatı haqqında dürüst qeyd

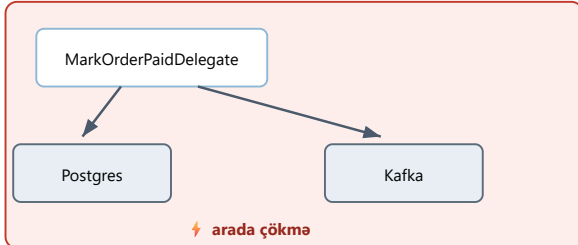
Normalda worker `/engine-rest` -i HTTP üzərindən poll edir (`camunda-external-task-client`) — məhz bu, onu istənilən dildə ayrıca deployment edə bilməyə imkan verir. Bu stack-də mümkün olmadı: Camunda 7.24-ün REST starter-i **Jersey** üzərində qurulub, Spring Boot 4 isə Jersey dəstəyini tamamilə çıxarıb. Ona görə worker engine-side `ExternalTaskService` -i birbaşa çağırır. Əməliyyatlar eynidir — REST client məhz bunları wrap edir — yəni çatışmayan şəbəkə hop-udur, pattern deyil.

11. Bənd 7 — Kafka və Transactional Outbox

Dual write problemi

Ən aşkar həll `MarkOrderPaidDelegate` -in içindən Kafka producer-ini çağırmaqdır. Bu, **dual write**-dir: engine sifariş və proses vəziyyətini Postgres-ə commit edir, producer isə broker-ə göndərir, və bu iki sistemi birləşdirən tranzaksiya *yoxdur*.

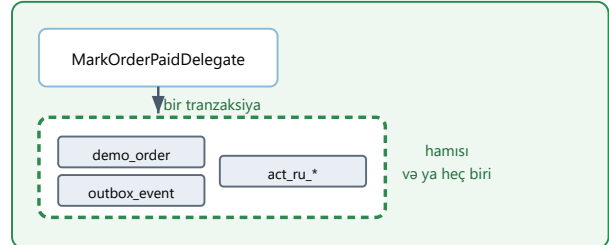
✗ Dual write



- Commit oldu, göndərmə olmadı → **event itdi**
- Göndərildi, commit rollback → **olmayan hadisə**

Çağırışların sırasını dəyişmək bunu HƏLL ETMİR.

✓ Transactional outbox



Commit-dən SONRA:



Qalan tək zəiflik: ack ilə publishedAt yenilənməsi
hələ də iki sistemdir → **at-least-once**.
Təkrarlamaq itirməkdən yaxşıdır → consumer idempotent olmalıdır.

Şəkil 8 — Outbox ikinci sistemi kritik yoldan çıxarır, qalan risk şüurlu şəkildə seçilmiş at-least-once-dir.

Yazma tərəfi

MARKORDERPAIDDELEGATE.JAVA

```
orderRepository.findByCorrelationId(correlationId).ifPresent(order -> {
    order.markPaid();
    orderRepository.save(order);

    // Status dəyişikliyi və proses vəziyyəti ilə EVNİ tranzaksiyada.
    // Event nə rollback-dan sağ çıxa, nə də commit-dən sonra itə bilər.
    outbox.record(OrderEventOutbox.ORDER_PAID, correlationId,
        order.getCustomerName(), order.getAmount());
});
```

Göndərmə tərəfi

OUTBOXPUBLISHER.JAVA

```
@Scheduled(fixedDelayString = "${demo.outbox.poll-interval-ms:1000}")
@Transactional
public void publishPending() {
    List<OutboxEvent> pending = outboxEventRepository.findTop100ByPublishedAtIsNullOrderByIdAsc();

    for (OutboxEvent event : pending) {
        try {
            // Ack-i gözləmək vacibdir: onsuz sətri "göndərilib" işarələyərdik,
            // halbuki record hələ producer buferindədir.
            kafkaTemplate.send(topic, event.getAggregateId(), event.getPayload())
                .get(ACK_TIMEOUT_SECONDS, TimeUnit.SECONDS);
            event.markPublished();
        } catch (Exception e) {
            // İlk xətdə DAVANIRIQ, keçib getmirik – əks halda eyni sifarişin
            // gec event-i erkənindən əvvəl gedər və keying-in mənası itər.
            return;
        }
    }
}
```

Qərar	Səbəb
Kafka key = <code>correlationId</code>	Bir sifarişin bütün event-ləri eyni partition-a düşür → sıra qorunur . Consumer həm də bu açarla dedup edə bilir.
<code>acks: all</code>	Bütün in-sync replika yazana qədər gözlə. Zəif ayar "published" işarələnmiş, amma davamlı saxlanmamış sətirlər deməkdir.
İlk xətdə dayan	Sıranın pozulmaması üçün.
Əvvəl göndər, sonra işarələ	Tərsi event <i>itirir</i> . Təkrar göndərmək təhlükəsizdir, itirmək deyil.

Oxuma tərəfi — at-least-once ilə yaşamaq

PAYMENTEVENTCONSUMER.JAVA

```
@KafkaListener(topics = "${demo.kafka.payment-events-topic}",
    groupId = "${demo.kafka.consumer-group}")
public void onPaymentEvent(String correlationId) {
    Optional<Order> order = orderService.findByCorrelationId(correlationId);

    if (order.isEmpty()) {
        // Gözləmək sifariş yaratmır → retry mənasızdır.
        throw new UnknownOrderException(correlationId);    // non-retryable → DLT
    }

    if (order.get().getStatus() != OrderStatus.AWAITING_PAYMENT) {
        // TƏKRAR çatdırılma. Status dedup açağıdır.
        return;    // no-op
    }

    if (!orderService.notifyPaymentReceived(correlationId)) {
        // Korrelyasiya xətası adətən "səhv" yox, "TEZ" deməkdir:
        // instansiya hələ mesaj event-inə çatmayıb (məsələn manual review-dadır).
        throw new NotWaitingForPaymentException(correlationId); // retryable
    }
}
```

Vəziyyət	Reaksiya	Səbəb
Sifariş artıq ödənilib	<code>no-op</code>	Burada xəta atmaq sonsuz retry deməkdir
Manual review-dadır	<code>throw → retry</code>	Proses sağdır, ödənişi sonra qəbul edəcək
Bela sifariş yoxdur	<code>throw → DLT</code>	Gözləmək sifariş yaratmır

DLT olmasa nə olur?

Daimi uğursuz record sonsuz retry olunur və **öz partition-unu bloklayır**. Onun arxasındakı bütün düzgün event-lər dayanır. Bir "zəhərli" mesaj bütün növbəni dayandırır.

KAFKACONFIG.JAVA

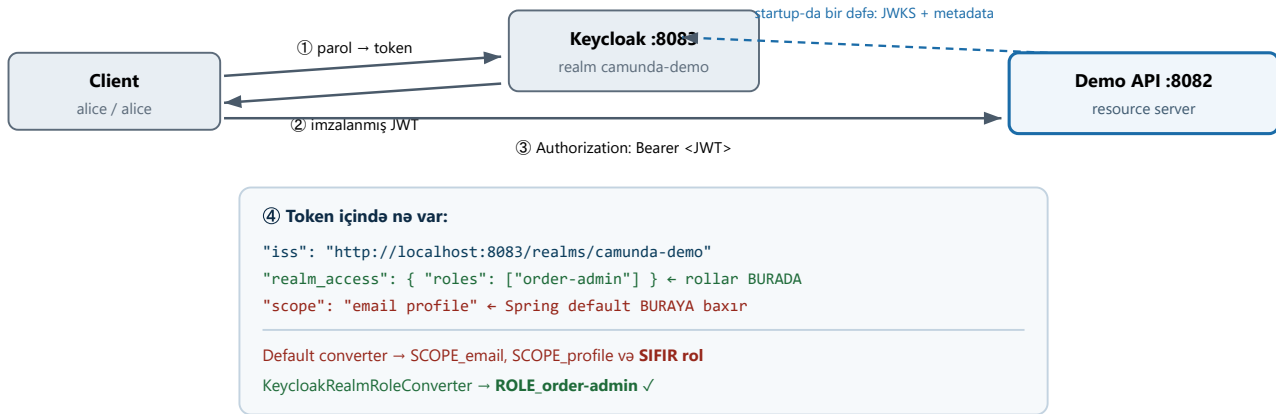
```
@Bean
DefaultErrorHandler kafkaErrorHandler(KafkaTemplate<String, String> kafkaTemplate,
    @Value("${demo.kafka.payment-events-dlt-topic}") String deadLetterTopic) {
    // Hədəf ADLA verilir, recoverer-in default suffiks konvensiyasına buraxılıdır –
    // elan olunan topic ilə həqiqətən yazılan topic ayrılmalı bilməsin.
    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(
        kafkaTemplate, (record, exception) -> new TopicPartition(deadLetterTopic, -1));

    DefaultErrorHandler handler = new DefaultErrorHandler(recoverer, new FixedBackOff(2000L, 3));
    handler.addNotRetryableExceptions(PaymentEventConsumer.UnknownOrderException.class);
    return handler;
}
```

12. Bənd 8 — OAuth2 / OIDC

Resource server nədir?

/api/** **resource server**-dir: gələn bearer token-i Keycloak-ın dərc etdiyi imza açarları ilə yoxlayır və içindəki rollara görə icazə verir. Bu tətbiqə heç vaxt parol çatmır və hər sorğuda Keycloak-a gediş-gəliş olmur — imza və `iss / exp` claim-ləri kifayətdir.



Şəkil 9 — Token axını və "Keycloak rollarını işləmir" probleminin asl səbəbi.

Ən çox rast gəlinən səhv

Rollar `scope`-da deyil

Spring Security-nin default `JwtGrantedAuthoritiesConverter`-i `scope / scp` claim-inə baxır. Keycloak isə realm rollarını `realm_access.roles`-a qoyur. Nəticədə **tamamilə etibarlı token** heç bir rol vermir və bütün `hasRole` yoxlamaları uğursuz olur — bu isə token problemi kimi görünür, halbuki mapping problemidir.

KEYCLOAKREALMROLECONVERTER.JAVA

```
public class KeycloakRealmRoleConverter
    implements Converter<Jwt, Collection<GrantedAuthority>> {

    @Override
    public Collection<GrantedAuthority> convert(Jwt jwt) {
        Map<String, Object> realmAccess = jwt.getClaimAsMap("realm_access");

        if (realmAccess == null
            || !(realmAccess.get("roles") instanceof Collection<?> roles)) {
            return List.of(); // rolsuz istifadəçi = 403, 500 deyil
        }

        return roles.stream()
            .map(String::valueOf)
            .map(role -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" + role))
            .toList();
    }
}
```

Client rolları isə tamamilə başqa yerdədir: `resource_access.<clientId>.roles`.

Üç filter chain, bir yox

Səth	CSRF	Səbəb
/api/**	söndürülüb	Stateless bearer token — session yox, cookie yox, deməli minmək üçün heç nə yoxdur. Yeganə söndürüldüyü yer.
/camunda/**	söndürülüb	Camunda öz login-i və öz CSRF filter-i ilə gəlir; üstünə Spring-inkini qoymaq onun POST-larını rədd edir.
qalan hər şey	aktiv	Demo UI açıqdır, amma CSRF qalır. Thymeleaf hər <code>th:action</code> formuna token əlavə edir.

SECURITYCONFIG.JAVA

```
@Bean
@Order(1)
SecurityFilterChain apiSecurityFilterChain(HttpSecurity http,
                                           JwtAuthenticationConverter converter) throws Exception {
    return http
        .securityMatcher("/api/**")
        .authorizeHttpRequests(a -> a
            .requestMatchers(HttpMethod.POST, "/api/orders").hasRole("order-admin")
            .requestMatchers(HttpMethod.POST, "/api/orders/*/payment").hasRole("order-admin")
            .requestMatchers(HttpMethod.GET, "/api/orders/**")
                .hasAnyRole("order-admin", "order-viewer")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(j -> j.jwtAuthenticationConverter(converter)))
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(csrf -> csrf.disable())
        .build();
}
```

issuer-uri VS jwk-set-uri

Ayar	Nə edir
issuer-uri production	Startup-da OIDC metadata-nı çəkir, JWKS ünvanını oradan alır. iss claim-ini də yoxlayır — başqa realm-dan gələn düzgün imzalanmış token rədd olunur.
jwk-set-uri test	Yalnız açar dəstini göstərir, startup-da şəbəkəyə çıxmır. Testlərdə istifadə olunur ki, hər test qaçışı üçün Keycloak lazım olmasın.

401 vs 403 — canlı yoxlanmış matris

Çağıran	POST /api/orders	Mənası
token yox	401	Kim olduğunu bilmirik
səhv formatlı token	401	Autentifikasiya uğursuz
carol — rolsuz	403	Kim olduğunu bilirik, icazəsi yoxdur
bob — order-viewer	403	Oxuya bilər, yazı bilməz
alice — order-admin	200	İcazəlidir

Nəyi qəsdən daxil etmədim

Cockpit/Tasklist-ə **SSO** ayrı işdir: `ContainerBasedAuthenticationFilter` plus OIDC principal-ı Camunda-nın identity service-inə map edən `AuthenticationProvider`, ya da community Keycloak identity plugin-i lazımdır. Həmin plugin Spring Boot 3 üçündür, bu layihə isə Boot 4-dədir — və Boot 4 bu repoya artıq iki dəfə bəla oturub (Jersey, Kafka autoconfig). Yarımqıq bağlamaqdansa kənarda saxlanıb.

13. Yol boyu qarşılaşdığım real problemlər

Bunlar sənəddə yazılmayan, yalnız işlədəndə üzə çıxan şeylərdir. Müsahibədə "hansı çətinliklə qarşılaşdınız?" sualına konkret cavabdır.

13.1 Spring Boot 4 — Jersey silinib

Simptom

`camunda-bpm-spring-boot-starter-rest` əlavə edən kimi **bütün** Spring context yüklənməsi çökdü:

STDERR

```
ClassNotFoundException:
    org.springframework.boot.autoconfigure.web.servlet.JerseyApplicationPath
Error processing condition on CamundaBpmWebappAutoConfiguration
```

Səbəb: Camunda 7.24-ün REST starter-i Jersey üzərində qurulub. Spring Boot 4 Jersey auto-configuration-ı tamamilə çıxarıb. Yəni bu stack-də `/engine-rest` -i qaldırmaq mümkün deyil.

Həll: External task worker engine-side `ExternalTaskService` -i birbaşa çağırır. Pattern qorunur, yalnız HTTP hop-u yoxdur. Bu, `build.gradle` -də və README-də açıq sənədləşdirilib.

13.2 Spring Boot 4 — Kafka auto-config ayrı modula köçüb

`org.springframework.kafka:spring-kafka` əlavə etmək kifayət etmədi: `KafkaTemplate` bean-i yox idi, `@KafkaListener` işləmirdi, `spring.kafka.*` property-ləri oxunmurdu.

Səhv	Düzgün
<code>org.springframework.kafka:spring-kafka</code>	<code>org.springframework.boot:spring-boot-starter-kafka</code>
<code>org.springframework.boot.autoconfigure.kafka.KafkaProperties</code>	<code>org.springframework.boot.kafka.autoconfigure.KafkaProperties</code>
<code>org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc</code>	<code>org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc</code>

13.3 `KafkaTemplate<String,String>` inject olunmur

Boot-un auto-config etdiyi bean `KafkaTemplate<?, ?>` kimi elan olunub və `KafkaTemplate<String, String>` tələb edən injection point-ə uyğun gəlmir: `NoSuchBeanDefinitionException`. Bu, **production-da da sınırdı** — testlər sayəsində erkən tapıldı.

KAFKACONFIG.JAVA - HƏLL

```
@Bean
KafkaTemplate<String, String> kafkaTemplate(KafkaProperties kafkaProperties) {
    return new KafkaTemplate<>{
        new DefaultKafkaProducerFactory<>(kafkaProperties.buildProducerProperties());
    }
}
// Öz bean-imiz olduğu üçün Boot-un @ConditionalOnMissingBean tərifi geri çəkilir.
```

13.4 DLT adı uyğunsuzluğu — yalnız canlı yoxlamada üzə çıxdı

Testlər bunu tuta bilməzdi

`payment-events.DLT` adlı topic elan etmişdim. Spring Kafka 4-ün default konvensiyası isə `<topic>-dlt` -dir. Nəticədə record auto-create olunmuş `payment-events-dlt` -yə düşürdü, mənim elan etdiyim topic isə **boş qalırdı**. Log-dakı ipucu:

APP LOG

```
WARN DeadLetterPublishingRecoverer : Destination resolver returned
non-existent partition payment-events-dlt-0
```

Həll: hədəfi konvensiyaya buraxmaq əvəzinə açıq təyin etmək.

13.5 `ddl-auto: update` CHECK constraint-i yeniləmir

`OrderStatus` enum-una `REFUNDED` əlavə etdim. UI-də "Simulate Payment" 500 verdi:

STDERR

```
new row for relation "demo_order" violates check constraint "demo_order_status_check"
```

Hibernate `update` rejimində **mövcud CHECK constraint-i dəyişmir** — o, yalnız köhnə üç dəyərə yaradılmışdı. Əl ilə düzəldildi:

SQL

```
ALTER TABLE demo_order DROP CONSTRAINT demo_order_status_check;  
ALTER TABLE demo_order ADD CONSTRAINT demo_order_status_check  
CHECK (status IN ('AWAITING_PAYMENT', 'PAID', 'CANCELLED', 'REFUNDED'));
```

Dərs

Bu, real layihələrdə `ddl-auto` yerinə **Flyway/Liquibase** istifadə etməyin klassik səbəbidir. Müsahibədə soruşulsa, konkret nümunəniz var.

13.6 Event-based gateway-in activity id-si

`isWaitingAt("Event_PaymentReceived")` uğursuz oldu. `ActivityInstance` ağacını çap edəndə məlum oldu ki, gateway arxasında park olunmuş execution-lar **gateway-in öz id-si** (`Gateway_WaitForEvent`) ilə görünür — fərdi catch event-lərin id-si ilə deyil, ta ki budaqlardan biri həqiqətən işə düşənə qədər.

13.7 BPMN fayl "özbaşına" geri qayıtdı

Gateway və user task əlavə etdikdən sonra UI-də 6000 dəyəri review task yaratmadı. Araşdırma göstərdi ki, deploy olunmuş versiyada gateway yoxdur — sonra isə `grep` sübut etdi ki, **faylın özü diskdə geri qayıdıb** (yəqin IDE bağlananda köhnə buferi yazıb).

Nəticə

Məhz bu hadisə `git init`-ə səbəb oldu. Versiya nəzarəti olmadan belə bir şeyi aşkarlamaq və bərpa etmək çox baha başa gəlir.

14. Müsahibə üçün sual-cavab

Camunda əsasları

Sual	Qısa cavab
Embedded və standalone engine fərqi?	Embedded — engine tətbiqin JVM-ində, tranzaksiya paylaşılır, delegate-lər sizin bean-lərinizdir. Standalone — ayrıca server, REST ilə müraciət, delegate-lər serverin classpath-ində olmalıdır (ya da external task).
Wait state nədir?	Engine-in tranzaksiyanı commit edib dayandığı nöqtə: user task, mesaj/timer catch event, external task, async continuation.
Business key nə üçündür?	Proses instansiyasını biznes obyektinə bağlamaq. Cockpit-də axtarış, <code>processInstanceBusinessKey()</code> sorğuları.
Message correlation necə işləyir?	<code>act_ru_event_subscr</code> -də abunəliklər saxlanılır; korrelyasiya dəyişən/business key üzrə indeksli axtarışdır. <code>correlate</code> , <code>correlateAll</code> , <code>correlateExclusively</code> fərqləri.

Xəta idarəetməsi

Sual	Qısa cavab
<code>BpmnError</code> nə vaxt, exception nə vaxt?	<code>BpmnError</code> — modeləşdirilmiş biznes nəticəsi, boundary error event tutur. Exception — texniki nasazlıq, engine retry edir, sonra incident. Texniki xətanı kompensasiya etmək = şəbəkə problemi ucbatından müştəriyə pul qaytarmaq.
Incident nədir və nə vaxt yaranır?	Job-un retry-ları 0-a çatanda <code>act_ru_incident</code> -ə sətir yazılır. Proses ölmür — park olunur. Operator Cockpit-də səbəbi düzəldib retry-ı sıfırlayır.
Saga pattern-i Camunda-da necə qurulur?	Kompensasiya olunan task-a <code>compensateEventDefinition</code> -li boundary event, <code>isForCompensation="true"</code> handler, aralarında <code><association></code> , və tetikləmək üçün <code>intermediateThrowEvent</code> .
Kompensasiya rollback-dan nə ilə fərqlənir?	Rollback commit olunmamış dəyişikliyi silir, iz qalmır. Kompensasiya artıq commit olunmuş və görünmüş addımı əks əməliyyatla balanslaşdırır — auditdə iz qalır (PAID → REFUNDED, AWAITING_PAYMENT deyil).

Engine daxili

Sual	Qısa cavab
<code>asyncBefore</code> nə üçündür?	Tranzaksiya sərhədi. Engine burada commit edir və qalanını job-a köçürür. Sonrakı addım çökməyə əvvəlki commit itmir.
<code>asyncBefore</code> vs <code>asyncAfter</code> ?	<code>Before</code> — activity-dən əvvəl commit; activity-nin özü retry olunur. <code>After</code> — activity-dən sonra; activity təkrar icra olunmur, yalnız sonrakı keçid.
<code>lock-time-in-millis</code> niyə vacibdir?	Ən yavaş job-dan qısa olsa, job hələ işləyərkən başqa node onu götürür və iki dəfə icra olunur .
Klasterdə <code>OptimisticLockingException</code> ?	Normaldır. Node-lar eyni job-u kilidləməyə çalışır; engine həmin dövrəni buraxır. Çox görünürsə <code>max-jobs-per-acquisition</code> böyükdür.
Job executor pool ölçüsü nəyə görə seçilir?	Hikari connection pool-dan böyük olmamalıdır — əks halda darboğaz sadəcə thread-lərdən DB bağlantılarına köçür.

Versiyalaşma və deployment

Sual	Qısa cavab
Yeni versiya deploy edəndə işləyən instansiyalara nə olur?	Heç nə. Onlar başladılan definition-a bağlı qalır. Yalnız yeni instansiyalar yeni versiyanı götürür.
Onları necə köçürsünüz?	<code>createMigrationPlan(src, tgt).mapEqualActivities().build()</code> + <code>newMigration(plan).processInstanceIds(...).execute()</code> . Toplu üçün <code>.processInstanceQuery(...)</code> və <code>.executeAsync()</code> .
Activity id dəyişibse?	Plan qurulur, amma <code>execute()</code> <code>MigratingProcessInstanceValidationException</code> atır və instansiya toxunulmamış qalır. Açıq <code>.mapActivities("Old", "New")</code> lazımdır.
Timer müddəti dəyişibse?	<code>.updateEventTrigger()</code> — abunəlik yeni tərifə uyğun yenidən qurulur.

Inteqrasiya

Sual	Qısa cavab
Service task nə vaxt, external task nə vaxt?	Delegate — sürətli, JVM-daxili, engine tranzaksiyasında. External — uzun sürən iş, başqa dil/komanda, ayrıca scale, engine-dən asılı olmayan deployment. External task həm də wait state-dir: worker sönülü ikən heç nə xəta vermir.
<code>fetchAndLock</code> niyə lazımdır?	Çoxsaylı worker eyni topic-i poll edir; lock eyni işin iki dəfə görülməsinin qarşısını alır. Lock müddəti ən yavaş işdən uzun olmalıdır.
Retry-a kim qərar verir?	Delegate-də model (<code>failedJobRetryTimeCycle</code>). External task-da worker — <code>handleFailure(..., retriesLeft, backoff)</code> , yəni xətanın növünə görə fərqli qərar verə bilir.
Dual write problemi nədir?	DB commit-i və broker-ə göndərmə iki ayrı sistemdir, aralarında tranzaksiya yoxdur. Çökmə ya olmayan hadisə yaradır, ya da olmuş hadisəni itirir. Sıranı dəyişmək həll etmir.
Outbox necə həll edir?	Event sətiri status dəyişikliyi ilə <i>eyni tranzaksiyada</i> , eyni bazaya yazılır → atomikdir. Ayrıca publisher commit-dən sonra Kafka-ya göndərir. Qalan risk: ack ilə <code>publishedAt</code> arasında çökmə → at-least-once → consumer idempotent olmalıdır.
Niyə key = correlationId?	Bir sifarişin bütün event-ləri eyni partition-a düşür → sıra qorunur; consumer həm də bu açarlarla dedup edir.
DLT niyə lazımdır?	Daimi uğursuz record sonsuz retry olunub öz partition-unu bloklayır — arxasındakı bütün düzgün mesajlar dayanır.

Təhlükəsizlik

Sual	Qısa cavab
Keycloak rolları niyə işləmir?	Spring default converter-i <code>scope</code> -a baxır, Keycloak rolları <code>realm_access.roles</code> -a qoyur. Custom <code>Converter<Jwt, Collection<GrantedAuthority>></code> və <code>ROLE_</code> prefiksi lazımdır.
<code>issuer-uri</code> vs <code>jwk-set-uri</code> ?	<code>issuer-uri</code> OIDC discovery edir və <code>iss</code> claim-ini yoxlayır — başqa realm-in düzgün imzalanmış token-i rədd olunur. <code>jwk-set-uri</code> yalnız açarları göstərir.
401 və 403 fərqi?	401 — kim olduğunuzu bilmirik (autentifikasiya). 403 — bilirik, icazəniz yoxdur (avtorizasiya).
CSRF-i nə vaxt söndürmək olar?	Stateless bearer token API-də — session/cookie yoxdursa, minmək üçün heç nə yoxdur. Cookie-li session forması varsa söndürmək olmaz .
Camunda webapp-larına SSO necə edilir?	<code>ContainerBasedAuthenticationFilter</code> + OIDC principal-ı Camunda identity service-inə map edən <code>AuthenticationProvider</code> ; ya da community Keycloak identity plugin-i.

15. İşə salma təlimatı

Tələblər

Java 21, Docker, və layihənin öz Gradle wrapper-i.

1. Postgres

BASH

```
docker run -d --name camunda-postgres -p 5432:5432 \
-e POSTGRES_DB=camunda -e POSTGRES_USER=camunda -e POSTGRES_PASSWORD=camunda \
postgres:15
```

2. Kafka və Keycloak

BASH

```
docker compose up -d
```

Keycloak realm-i `keycloak/realm-camunda-demo.json` -dan import olunur — rollar, client və istifadəçilər daxil. Heç nə əl ilə klikləmək lazım deyil.

3. Tətbiq

BASH

```
./gradlew bootRun
```

Ünvan	Nə
<code>http://localhost:8082/orders</code>	Demo UI
<code>http://localhost:8082/camunda</code>	Cockpit / Tasklist / Admin — <code>demo</code> / <code>demo</code>
<code>http://localhost:8083</code>	Keycloak admin — <code>admin</code> / <code>admin</code>

4. Təhlükəsiz API-ni çağırmaq

BASH

```
TOKEN=$(curl -s -X POST \
  http://localhost:8083/realms/camunda-demo/protocol/openid-connect/token \
  -d grant_type=password -d client_id=camunda-demo-api \
  -d client_secret=demo-client-secret \
  -d username=alice -d password=alice | jq -r .access_token)

curl -X POST http://localhost:8082/api/orders \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"customerName":"Alice","amount":120}'
```

5. Kafka ilə ödəniş göndərmək

BASH

```
# Açar da, dəyər də correlationId-dir
docker exec -i camunda-demo-kafka /opt/kafka/bin/kafka-console-producer.sh \
--bootstrap-server localhost:9092 --topic payment-events \
--property parse.key=true --property key.separator=:
> <correlationId>:<correlationId>

# Prosesin nə yaydığına baxmaq
docker exec -i camunda-demo-kafka /opt/kafka/bin/kafka-console-consumer.sh \
--bootstrap-server localhost:9092 --topic order-events --from-beginning
```

6. Testlər

BASH

```
./gradlew test # 32 test, Docker tələb etmir - H2 + mock JWT
```

Demo ssenariləri

Müştəri adı / məbləğ	Nə göstərir
istənilən ad, < 1000	Normal axın: ödəniş → çatdırılma → invoice → tamamlandı
istənilən ad, >= 1000	DMN HIGH → manual review user task (Tasklist-də görünür)
ShipFail-...	BpmnError → kompensasiya → REFUNDED
TechFail-...	Texniki xəta → 3 retry → incident , status PAID qalır
ödəniş göndərmədən 1 saat	Timer → Cancel Order → CANCELLED

Yekun

32 test · 13 test klasi · 0 failure. Bütün mexanizmlər həm testlərdə, həm də canlı işləyən sistemdə (brauzer, real Kafka broker-i, real Keycloak token-ləri, DB sorğuları) yoxlanılıb.